

Three Fleet Scenarios for ePlanSys

EPDDL survey missions, grounded with plank and solved with Aletheia

Abstract

ePlanSys shipped one worked example, the muddy children puzzle. It is the right example for explaining what an epistemic goal *is* and the wrong one for a robotics user, who has no children and no mud. This note records three robot-fleet scenarios written to replace it, at three sizes, together with what Aletheia does on each. All three are checked into the ePlanSys tree as EPDDL sources with matching action mappings; the two smaller ones are also regression tests. The scenarios are deliberately the same three mechanisms at every size, namely drive, look and say what you saw, so that what grows between them is the search and not the modelling. Across the three, ground actions go from 8 to 48 and expanded nodes from 17 to 5.03 million, while peak resident memory moves from 4.34 MB to 4.85 MB.

1 Why a fleet

The muddy children puzzle is a good teaching instance and a poor advertisement. Nothing in it moves, nobody has a sensor, and the only action is a simultaneous public announcement that no robot performs. A reader wanting to know whether epistemic planning is worth the trouble on a real platform learns nothing from it about what an action costs, what a sensor observes, or what a radio does.

The scenarios below are built around a single situation that is common in fleet robotics and inexpressible in PDDL:

A corridor may be blocked. Only a robot standing at the junction can see it. The rest of the fleet has to route around it, so the rest of the fleet has to know, and watching a robot look tells you that it looked, not what it saw.

Three things follow, and they are the three mechanisms every scenario uses. *Driving* is ontic and public: it changes the world, and the fleet tracks its members' positions. *Looking* is semi-private sensing: the robot at the junction learns the answer and the others learn only that a look happened. *Reporting* is a public announcement guarded by the speaker's knowledge, so a robot can broadcast only what it actually found; because knowledge is factive, hearing it settles the matter for everyone.

The goal is always of the form $Kw_i(\text{blocked})$, knowing *whether* rather than knowing *that*. This is not a stylistic choice. A goal of $K_i(\text{blocked})$ is unachievable in the worlds where the corridor is clear, so a plan for it would be unexecutable about half the time.

2 The three scenarios

Corridor. Two robots, one route. Either could drive out and look, but looking is semi-private, so one robot sensing does not inform the other. Both could go and look, at four actions; one can go, look and broadcast, at three. The planner finds the second, which is to say it works out that talking is cheaper than everybody going to see for themselves.

Depot. Three robots, two routes, and a depot rule: a robot dispatched down a route stays on it. No robot can survey both routes, so the fleet must split the work, and the robot that never moves ends up knowing both answers purely by listening.

Survey. Four robots, three routes, same rules. Written to be big rather than clever: it adds no mechanism, only size. Eight worlds the fleet cannot tell apart to begin with, twelve goal conjuncts, and forty-eight ground actions.

Table 1: The three shipped fleet scenarios, as plank grounds them and as Aletheia solves them. Worlds is $|W|$ of the grounded initial model and all of them are designated, so every world is a live possibility at planning time. Goal conjuncts counts the Kw atoms of the goal. Times are single runs; see §4.

	Corridor	Depot	Survey
Robots	2	3	4
Routes	1	2	3
Atoms	3	8	15
Ground actions	8	24	48
Worlds (= designated)	2	4	8
Goal conjuncts	2	6	12
Solution depth	3	6	9
Policy leaves	2	4	8
Branches checked	4	12	28
Nodes expanded	17	1 670	5 029 063
Nodes generated	19	2 010	5 746 557
Search time (s)	0.00	0.01	58.82
Peak RSS (MB)	4.34	4.47	4.85

3 What the policies look like

The corridor policy is the smallest thing that is recognisably a contingent plan rather than a sequence:

```
goto-junction_r1
  inspect_r1
    report-blocked_r1 <- if the camera saw debris
    report-clear_r1 <- if it did not
```

The depot policy is the one worth reading twice, because the division of labour is not something the domain states anywhere:

```
goto-north_r1
  inspect-north_r1
    report-north-blocked_r1
      goto-south_r2
        inspect-south_r2
          report-south-blocked_r2
          report-south-clear_r2
        report-north-clear_r1
      goto-south_r2
        inspect-south_r2
          report-south-blocked_r2
          report-south-clear_r2
```

r1 takes the north route, r2 the south, and r3 never moves. Nothing assigns those roles; the search arrives at them because the depot rule makes any other allocation longer. r3 finishes the mission knowing the state of both corridors without having left the depot, which is the entire argument for modelling communication epistemically rather than as an ontic effect on a `told` predicate.

4 Method and environment

Every instance was ground with plank (`plank export -d ... -p ... -l intermediate.epddl`) and solved with the Aletheia binary (`epistemic_planner --task ... --plan ...`). Heuristic and strategy

were left to Aletheia’s selection policy in all three cases; it chose *knowledge-spread* under the rule *kw-only-goal* and *AO** under *sensing-small-designated* every time, which is the expected reading of a *Kw-only* goal over a small designated set. Every returned policy was accepted by Aletheia’s own validator.

Timings are single runs of `/usr/bin/time` on an otherwise idle machine: Intel Core i7-10870H at 2.20 GHz, 16 threads, 38 GB RAM, Ubuntu 22.04.5, kernel 6.8.0, GCC 11.4.0, Release build. They are not repeated measurements and should be read as orders of magnitude, not as benchmarks. The sub-10 ms figures in particular are at the resolution floor of the tool.

Versions. ePlanSys feature/epddl-front-end at 0050a87; Aletheia at d530d25; plank at a682480.

5 Observations

The search explodes and the representation does not. Expanded nodes grow by roughly two orders of magnitude per size step: 17, then 1 670, then 5.03 million, a factor of 98 and then 3011. Peak resident memory over the same range grows by 12 %, from 4.34 MB to 4.85 MB. This is the property Aletheia’s design targets: models are three flat bit matrices and closed lists store 128-bit fingerprints rather than states, so the cost of a large search is time rather than memory. A planner storing states outright would not have finished the third instance on this machine.

Depth is what hurts. The three instances differ in every parameter at once, but the one that tracks the cost is solution depth, which *AO** reaches by iterative deepening: 3, 6, 9. Each survey trip adds three actions to the depth and doubles the number of worlds, and the product of those two is what the $3011 \times$ jump is made of.

The grounder produces multi-pointed models. Before these scenarios, every task plank produced from the available EPDDL instances was single-pointed, with one designated world, so sensing had exactly one possible outcome and *AO** returned a chain. The only branching fixture in the ePlanSys tree was a file edited by hand. All three scenarios here ground to genuinely multi-pointed models, because their `:init` theories leave the corridors open rather than asserting a state for them. They are the first grounded branching instances in that tree, and the first that exercise the executor’s branch node on a policy nobody hand-wrote.

Both routes to the planner agree. ePlanSys reaches Aletheia two ways: `plansys2_epistemic_planner` links the search in, and `plansys2_aletheia_plan_solver` runs the binary as a subprocess and reads its plan file back. On the corridor and depot instances the in-process route reports node counts identical to the standalone binary, 17/19 and 1670/2010, and the subprocess route returns the same policy, translated through the action mapping into PlanSys2 action expressions. The two are the same planner and now have a shared test to say so.

6 Reproducing

```
# ground
plank export -d robot-fleet-depot-domain.epddl \
              -p robot-fleet-depot-problem.epddl \
              -l intermediate.epddl -o /tmp/out

# solve
epistemic_planner --task /tmp/out/robot-fleet-depot-problem.json \
                  --plan /tmp/plan.json --explain
```

Inside a built ePlanSys workspace the grounding step is implicit: the plan solver is given the .epddl sources directly:

```
planner:
  ros__parameters:
    EPISTEMIC:
      epddl_domain: ".../robot-fleet-depot-domain.epddl"
      epddl_problem: ".../robot-fleet-depot-problem.epddl"
      action_mapping: ".../mappings/robot-fleet-depot.json"
```

7 Limitations

The timings are single runs on one machine and the three instances differ in several parameters simultaneously, so nothing here isolates the effect of any one of them; the scaling study in the companion report does that properly. The survey instance takes about a minute, which is why it is shipped as an example rather than as a regression test; the corridor and depot instances carry that role. Nothing here has been executed on hardware: the policies are validated by Aletheia and translated into PlanSys2 action expressions, but the behaviour trees behind those expressions are stubs, so the claim is about planning and not about a fleet that drove anywhere.